

Confidential Hospital Records

NAME: DHARSHITHA.K
ROLL NO: 192521093
SLOT-C
CRYPTOGRAPHY

Problem Statement 3:

Confidential Hospital Records

A hospital wants to protect patient details such as patient ID, name, diagnosis, and prescription. Develop an application that encrypts the patient information before storing it and decrypts it only for authorized users.

Maintain confidentiality of patient data.

Problem Statement 3:

1. Explanation

- The objective of this application is to protect confidential patient information such as Patient ID, Patient Name, Diagnosis, and Prescription.
- The application encrypts the patient details before storing them in a text file.
- In this program, the XOR encryption technique is used with a secret key.
- The encrypted information is stored in a file named hospital_records.txt.
- Only an authorized user who provides the correct secret key can decrypt and view the original patient information.
- If an incorrect key is entered, access is denied.

Working Process

1. Enter the Patient ID.
2. Enter the Patient Name.
3. Enter the Diagnosis.
4. Enter the Prescription.
5. Enter a secret encryption key.
6. Encrypt the patient information using XOR.
7. Store the encrypted information in hospital_records.txt.
8. Enter the key to access the records.
9. If the key is correct, decrypt and display the patient details.
10. If the key is incorrect, unauthorized access is denied.

2. Input

Enter Patient ID: P101
Enter Patient Name: Anitha
Enter Diagnosis: Fever
Enter Prescription: Paracetamol
Enter Encryption Key (single character): H
Enter key to decrypt the record: H

3. C++ Code

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// XOR Encryption / Decryption Function
string encryptDecrypt(string text, char key)
{
    for (int i = 0; i < text.length(); i++)
    {
        text[i] = text[i] ^ key;
    }

    return text;
}

int main()
{
    string patientID, name, diagnosis, prescription;
    char key;

    // Input patient details
    cout << "Enter Patient ID: ";
    getline(cin, patientID);

    cout << "Enter Patient Name: ";
    getline(cin, name);

    cout << "Enter Diagnosis: ";
    getline(cin, diagnosis);

    cout << "Enter Prescription: ";
    getline(cin, prescription);

    cout << "Enter Encryption Key (single character): ";
    cin >> key;
```

```

// Encrypt patient information
string encID = encryptDecrypt(patientID, key);
string encName = encryptDecrypt(name, key);
string encDiagnosis = encryptDecrypt(diagnosis, key);
string encPrescription = encryptDecrypt(prescription, key);

// Store encrypted data in file
ofstream file("hospital_records.txt");

if (!file)
{
    cout << "Error opening file!" << endl;
    return 1;
}

file << encID << endl;
file << encName << endl;
file << encDiagnosis << endl;
file << encPrescription << endl;

file.close();

cout << "\nPatient record encrypted and saved successfully!"
    << endl;

// Decryption
char decryptKey;

cout << "\nEnter key to decrypt the record: ";
cin >> decryptKey;

if (decryptKey == key)
{
    ifstream inputFile("hospital_records.txt");

    string encryptedID;
    string encryptedName;
    string encryptedDiagnosis;
    string encryptedPrescription;

    getline(inputFile, encryptedID);
    getline(inputFile, encryptedName);
    getline(inputFile, encryptedDiagnosis);
    getline(inputFile, encryptedPrescription);
}

```

```

inputFile.close();

// Decrypt patient information
string decryptedID =
    encryptDecrypt(encryptedID, decryptKey);

string decryptedName =
    encryptDecrypt(encryptedName, decryptKey);

string decryptedDiagnosis =
    encryptDecrypt(encryptedDiagnosis, decryptKey);

string decryptedPrescription =
    encryptDecrypt(encryptedPrescription, decryptKey);

// Display decrypted information
cout << "\n--- Decrypted Patient Record ---"
    << endl;

cout << "Patient ID   : "
    << decryptedID << endl;

cout << "Patient Name   : "
    << decryptedName << endl;

cout << "Diagnosis     : "
    << decryptedDiagnosis << endl;

cout << "Prescription  : "
    << decryptedPrescription << endl;
}
else
{
    cout << "Unauthorized access! Invalid key."
        << endl;
}

return 0;
}

```

4. Output

The image shows a C++ IDE with two code snippets and their outputs. The first snippet is a decryption program, and the second is an encryption program. Both programs attempt to open a file, but both fail, resulting in an "Error opening file!" message.

Snippet 1: Decryption Program

```
16 {
38 // Ask for key to decrypt
39 char decryptKey;
40 cout << "\nEnter key to decrypt the note: ";
41 cin >> decryptKey;
42 // Check secret key
43 if (decryptKey == key)
44 {
45     ifstream inputFile("secure_notes.txt");
46     string encryptedData;
47     getline(inputFile, encryptedData);
48     inputFile.close();
49     // Decrypt the note
50     string decryptedNote =
51         encryptDecrypt(encryptedData, decryptKey);
52     cout << "\n--- Decrypted Note ---" << endl;
53     cout << decryptedNote << endl;
54 }
55 else
56 {
57     cout << "Unauthorized access! Invalid secret
58         key."
59         << endl;
60 }
61 return 0;
62 }
```

Output 1:

```
Enter your confidential note: My project presentation is
tomorrow
Enter Secret Key (single character): S
Error opening file!

=== Code Exited With Errors ===
```

Snippet 2: Encryption Program

```
15 {
27 cout << "Enter Encryption Key (single character): ";
28 cin >> key;
29 // Encrypt patient information
30 string encID = encryptDecrypt(patientID, key);
31 string encName = encryptDecrypt(name, key);
32 string encDiagnosis = encryptDecrypt(diagnosis,
33     key);
34 string encPrescription = encryptDecrypt
35     (prescription, key);
36 // Store encrypted data in file
37 ofstream file("hospital_records.txt");
38 if (!file)
39 {
40     cout << "Error opening file!" << endl;
41     return 1;
42 }
43 file << encID << endl;
44 file << encName << endl;
45 file << encDiagnosis << endl;
46 file << encPrescription << endl;
47 file.close();
48 cout << "\nPatient record encrypted and saved
49     successfully!"
50     << endl;
51 // Decryption
52 char decryptKey;
```

Output 2:

```
Enter Patient ID: P101
Enter Patient Name: Anitha
Enter Diagnosis: Fever
Enter Prescription: Paracetamol
Enter Encryption Key (single character): H
Error opening file!

=== Code Exited With Errors ===
```

```
15 {
53 {
69     string decryptedDiagnosis =
        decryptKey();
71     string decryptedPrescription =
72         encryptDecrypt(encryptedPrescription,
            decryptKey());
73     // Display decrypted information
74     cout << "\n--- Decrypted Patient Record ---"
75         << endl;
76     cout << "Patient ID   : "
77         << decryptedID << endl;
78     cout << "Patient Name  : "
79         << decryptedName << endl;
80     cout << "Diagnosis    : "
81         << decryptedDiagnosis << endl;
82     cout << "Prescription  : "
83         << decryptedPrescription << endl;
84 }
85 else
86 {
87     cout << "Unauthorized access! Invalid key."
88         << endl;
89 }
90 return 0;
91 }
```

Output

```
Enter Patient ID: P101
Enter Patient Name: Anitha
Enter Diagnosis: Fever
Enter Prescription: Paracetamol
Enter Encryption Key (single character): H
Error opening file!

=== Code Exited With Errors ===
```

5. File Storage

- The encrypted patient information is stored in:
- hospital_records.txt
- The original patient information is not directly stored as plain text.
- The information is converted into an encrypted form before being written to the file.

6. Security Features

- **Confidentiality:** Patient information is protected from unauthorized access.
- **Encryption:** XOR encryption is used to protect the data.
- **Secret Key:** A key is required to decrypt the records.
- **Access Control:** Incorrect keys are rejected.
- **Secure File Storage:** Encrypted patient data is stored in a text file.

7. Result

- Thus, the Confidential Hospital Records application was successfully implemented using C++. Patient ID, name, diagnosis, and prescription are encrypted before being stored in a file.
 - The original patient information can be viewed only by an authorized user with the correct secret key.
-